

Reviewer Pack — Minimal Number of Generators in Affine Quotient Groups and S...

Entry d9492f3e4702 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-04 17:32:26 UTC

Minimal Number of Generators in Affine Quotient Groups and SAT Clause Subset Entropy

Entry ID: d9492f3e4702 **Verdict:** INCONCLUSIVE **Recorded:** 2026-06-04 17:32:26 UTC

1. Conjecture

Field A (mathematical branch): Group Theory (Affine Quotient Groups) **Field B** (complexity object): Boolean Satisfiability (SAT) Clause Subsets

Statement:

For a given Boolean formula φ with m clauses, the minimal number of generators required to generate the affine quotient group $G(\varphi)$ is linearly correlated with its clause subset entropy $H(\varphi)$, such that the number of generators $n(G(\varphi)) = O(H(\varphi)^2)$.

Rationale (proposer's reasoning):

Affine quotient groups provide a structured way of encoding Boolean formulas, and their generator count could potentially reveal insights into the complexity of clause subsets. A higher clause subset entropy implies a more complex structure, which may require more generators to represent.

Taxonomy category: AffineQuotientGroup (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): dcc20ac7a3f352ee

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

For a Boolean formula φ with m clauses, if the minimal number of generators $n(G(\varphi))$ is less than or equal to $3 * H(\varphi)^2$ AND no seed produces an $n(G(\varphi)) > 10 * H(\varphi)^2$, then support for the conjecture is provided. If any seed yields an $n(G(\varphi))$ greater than $10 * H(\varphi)^2$ OR if $n(G(\varphi))$ exceeds $3 * H(\varphi)^2$ for any seed, the conjecture is falsified.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	1.00	UNCERTAIN	SAFE
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - "affine quotient groups" AND "Boolean satisfiability" AND "generators"
- "entropy in group theory" AND "SAT clause subsets" AND "minimal generators" - "group theory"
~ " Boolean satisfiability" [field] AND generator count ~ entropy [term]

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_formula(n, m):
        variables = [f'x{i}' for i in range(n)]
        clauses = []
        for _ in range(m):
            clause = random.sample(variables, 2)
            if random.choice([True, False]):
                clause = [-1 * int(x[1:]) if x.startswith('-') else int(x) for x in clause]
            clauses.append(clause)
        return clauses

    def affine_quotient_group_size(clauses):
        n = len(set(var for clause in clauses for var in clause))
        m = len(clauses)
        return 2**n * (m + 1)

    def entropy(clauses):
        n = len(set(var for clause in clauses for var in clause))
        m = len(clauses)
        if m == 0:
            return 0
        p = m / (2**n * (m + 1))
        return -p * math.log2(p) - (1 - p) * math.log2(1 - p)

    def min_generators(n, m):
        # Simplified heuristic for the minimal number of generators
        return n
```

```

n_max = 40
instances_tested = 30
metric_values = []
conjecture_holds = True
counterexample = ""

for _ in range(instances_tested):
    n = random.randint(5, n_max)
    m = random.randint(1, n * 2)
    clauses = generate_formula(n, m)
    generators = min_generators(n, m)
    H_phi = entropy(clauses)

    if generators > 3 * H_phi**2:
        conjecture_holds = False
        counterexample = f"n={n}, m={m}, H( $\phi$ )={H_phi}, n(G( $\phi$ ))={generators}"
        break

    if generators > 10 * H_phi**2:
        conjecture_holds = False
        counterexample = f"n={n}, m={m}, H( $\phi$ )={H_phi}, n(G( $\phi$ ))={generators}"
        break

    metric_values.append(generators)

return {
    "metric_name": "Minimal Number of Generators",
    "metric_value": sum(metric_values) / len(metric_values),
    "instances_tested": instances_tested,
    "n_max": n_max,
    "conjecture_holds": conjecture_holds,
    "counterexample": counterexample
}

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] if sys.argv[1:] else [2, 3, 5, 7, 11, 13, 17, 19, 23, 29]

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_value = sum(r["metric_value"] for r in results) / len(results)
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_value} std=0.0 support_fraction={support_fraction}")
    elif any(r["counterexample"] for r in results):
        first_failing_seed = next((r["seed"] for r in results if r["counterexample"]), None)
        print(f"RESULT: FALSIFIED counterexample={results[0]['counterexample']} first_failing={first_failing_seed}")
    else:
        print("RESULT: INCONCLUSIVE no seeds supported the conjecture")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

--STDERR--

Traceback (most recent call last):

```
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_d52386da.py", line 88, in <module>
    result = run_trial(seed)
             ^^^^^^^^^^^^^
```

```
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_d52386da.py", line 57, in run_trial
    clauses = generate_formula(n, m)
              ^^^^^^^^^^^^^^^^^^^
```

```
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_d52386da.py", line 27, in generate_form
    clause = [-1 * int(x[1:]) if x.startswith('-') else int(x) for x in clause]
              ^^^^^^
```

ValueError: invalid literal for int() with base 10: 'x28'

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test code crashed before producing data, which prevents a clear evaluation of the conjecture. | next: Investigate and fix the crash in the test code to proceed with the verification of the conjecture.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	14388
2	preregistration	ollama_remote	glm4:latest	0	0	10183
3	novelty	ollama_remote	glm4:latest	0	0	8508
4	novelty	ollama_remote	glm4:latest	0	0	15325
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	13518
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	9394
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	9530
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	9823
9	judge	ollama_remote	glm4:latest	0	0	11982

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 102649 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in [research/audit/d9492f3e4702.jsonl](#))

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/d9492f3e4702.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/d9492f3e4702.tar.gz` (if generated)